Lab 04 - GPG Key Management, Encryption and Signing

Student: Nenad Damjanović | Environment: Kali Linux (VirtualBox)

Objective and environment

Use GNU Privacy Guard (GPG 2.4.9) to create asymmetric keys, export/import public keys, encrypt/decrypt messages, and create/verify digital signatures. The exercise also tests integrity after file modification and simulates secure correspondence with Alice.

Key generation and revocation

Generated an RSA 4096-bit signing/certification key with an RSA encryption subkey for nenad@test.com. The key was created on 19 October 2026 and expires on 20 October 2026; its expiration was intentionally left unchanged. A previously expired Ed25519 laboratory key was separately revoked.

Figure 1. GPG version and RSA key generation



Keyring inspection and public-key export

The gpg --list-keys and gpg --list-secret-keys outputs show the new RSA key and the revoked old key. The command gpg --armor --export nenad@test.com > public_key.asc created a distributable ASCII-armored public key.

Figure 2. Public and private key listing; revoked former key

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --list-keys
/home/kali/.gnupg/pubring.kbx
------------------------------
pub   ed25519 2026-10-14 [SC] [revoked: 2026-10-19]
      632CC7ADB2F5483E18F596421A5F761EC7D41D33
uid           [ revoked] Nenad Lab (SEEK AND DESTROY) <nenad.lab@example.com>

pub   rsa4096 2026-10-19 [SC] [expires: 2026-10-20]
      1F5E9B5C5C53A8941CE23AC8692B31616BFCB6C5
uid           [ultimate] Neand Damjanović <nenad@test.com>
sub   rsa4096 2026-10-19 [E] [expires: 2026-10-20]


┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --list-secret-keys
/home/kali/.gnupg/pubring.kbx
------------------------------
sec   ed25519 2026-10-14 [SC] [revoked: 2026-10-19]
      632CC7ADB2F5483E18F596421A5F761EC7D41D33
uid           [ revoked] Nenad Lab (SEEK AND DESTROY) <nenad.lab@example.com>

sec   rsa4096 2026-10-19 [SC] [expires: 2026-10-20]
      1F5E9B5C5C53A8941CE23AC8692B31616BFCB6C5
uid           [ultimate] Neand Damjanović <nenad@test.com>
ssb   rsa4096 2026-10-19 [E] [expires: 2026-10-20]
```

Figure 3. Public-key export and self-encryption

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --armor --export nenad@test.com > public_key.asc

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ ls -lh public_key.asc
-rw-rw-r-- 1 kali kali 3.1K Oct 19 18:22 public_key.asc

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ head -n 5 public_key.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGrWPyQBEADJJIYiGHad+s9+6nIF6CBKZvSksDpYmkYCwcrDfeRGAinB1zfL
S1ArOIDkvPKYBtzMEGs714dFnRfPQO0c/c6ud3cHQ3QSLznFe2bASLEgzDLyGnbA
tRWHUSmk6TCG7GMZuiDkjBoWidLl8EQHm+kiyn2ZLrwuFsQzTQDvPBJpeE7iFu5w

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ echo "This is a confidential message for Lab 04." > message.txt

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --armor --encrypt \
  --recipient nenad@test.com \
  --output message_encrypted.asc \
  message.txt

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ ls -lh message*
-rw-rw-r-- 1 kali kali 923 Oct 19 18:24 message_encrypted.asc
-rw-rw-r-- 1 kali kali  43 Oct 19 18:24 message.txt

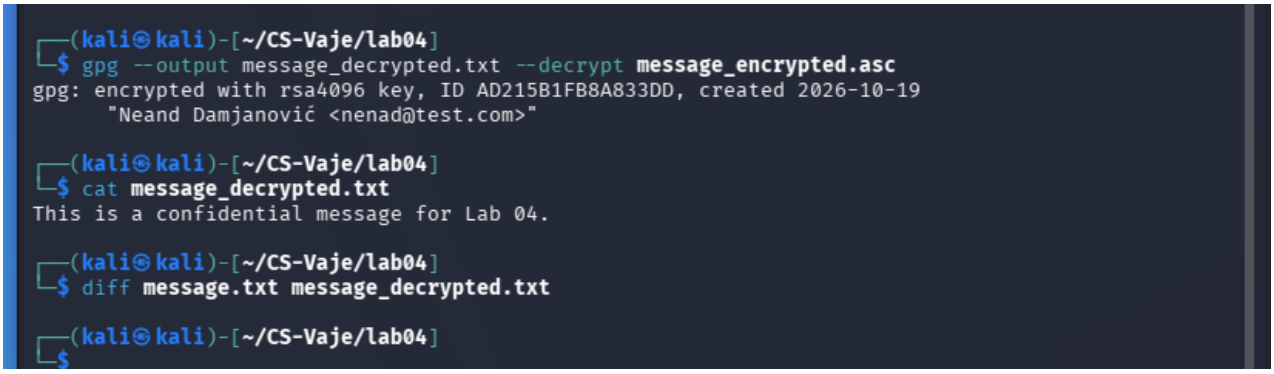
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ head -n 5 message_encrypted.asc
-----BEGIN PGP MESSAGE-----

hQIMA60hWx+4qDPdARAAm+wnlPahMebTiXawLK5GMc6SUTmUHWIFAuPnovi9UXcU
a8lLu/xHTURAtNAtDTmDvqdYoivULoilW1PQfq2ElJBTHeZLnhToR7V4AA+IJU4U
2kRuw9giNDsqO1YLTgSarcQf5i/7R2qkeNqEy+kvqXpCcZujIX2O51uuf72FUTB3
```

Encryption, decryption and signature verification

A test message was encrypted to the owner’s public key and decrypted using the corresponding private key. diff produced no output, proving that the recovered plaintext matched the original.

Figure 4. Successful decryption and file comparison



The original text was signed using a detached GPG signature. Verification reported Good signature. After modifying a copy of the text, verifying the original signature against the changed copy reported BAD signature. This demonstrates detection of tampering, not prevention of modification.

Figure 5. Valid signature on the original message

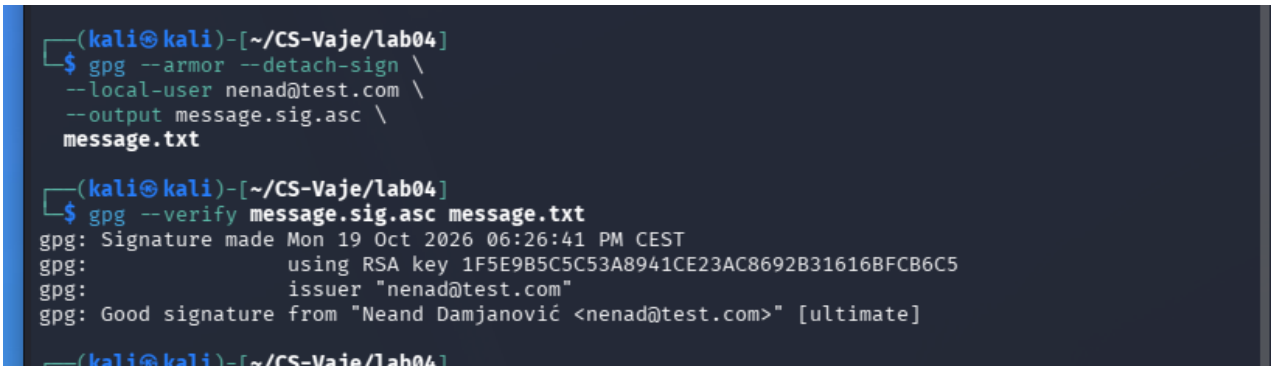
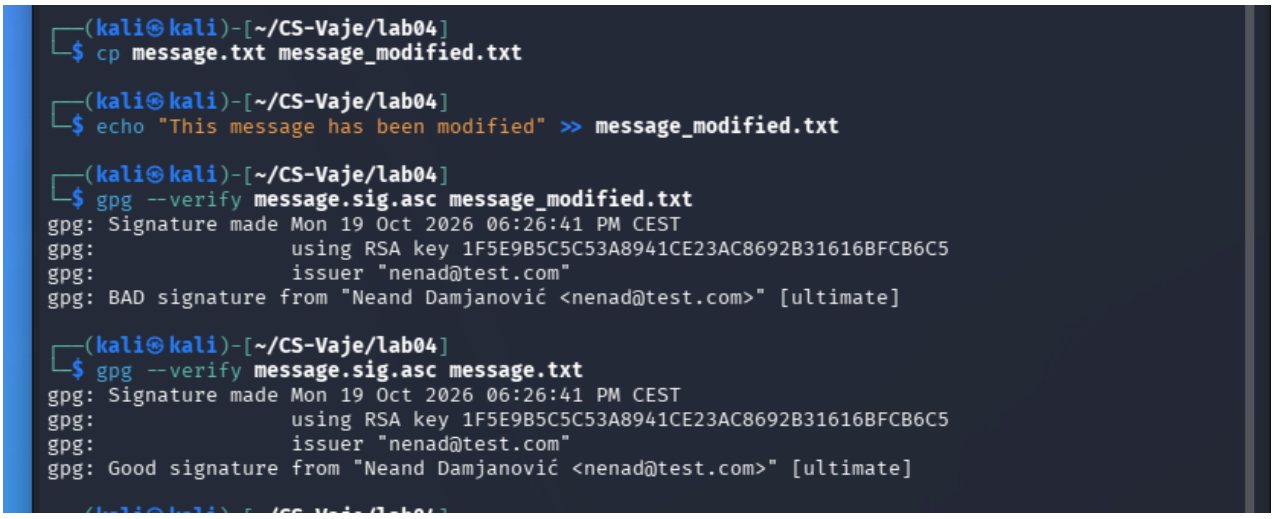


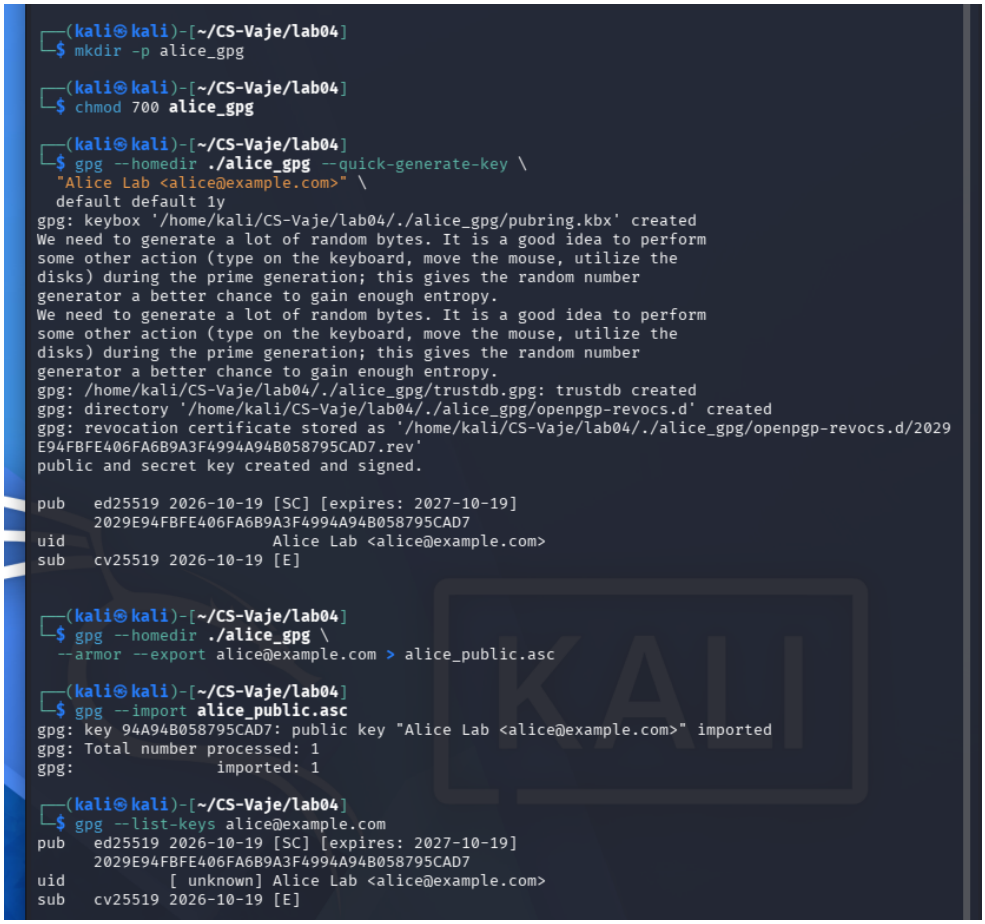
Figure 6. Invalid signature for altered data; valid for the original



Exchange of public keys and signed encryption

To simulate a separate recipient, an isolated alice_gpg keyring was created. Alice's public key was exported and imported into Nenad's keyring; the private key remained in Alice's isolated directory.

Figure 7. Alice's key creation and import of her public key



A message for Alice was encrypted using her public encryption key and simultaneously signed using Nenad's private signing key (gpg --armor --encrypt --sign).

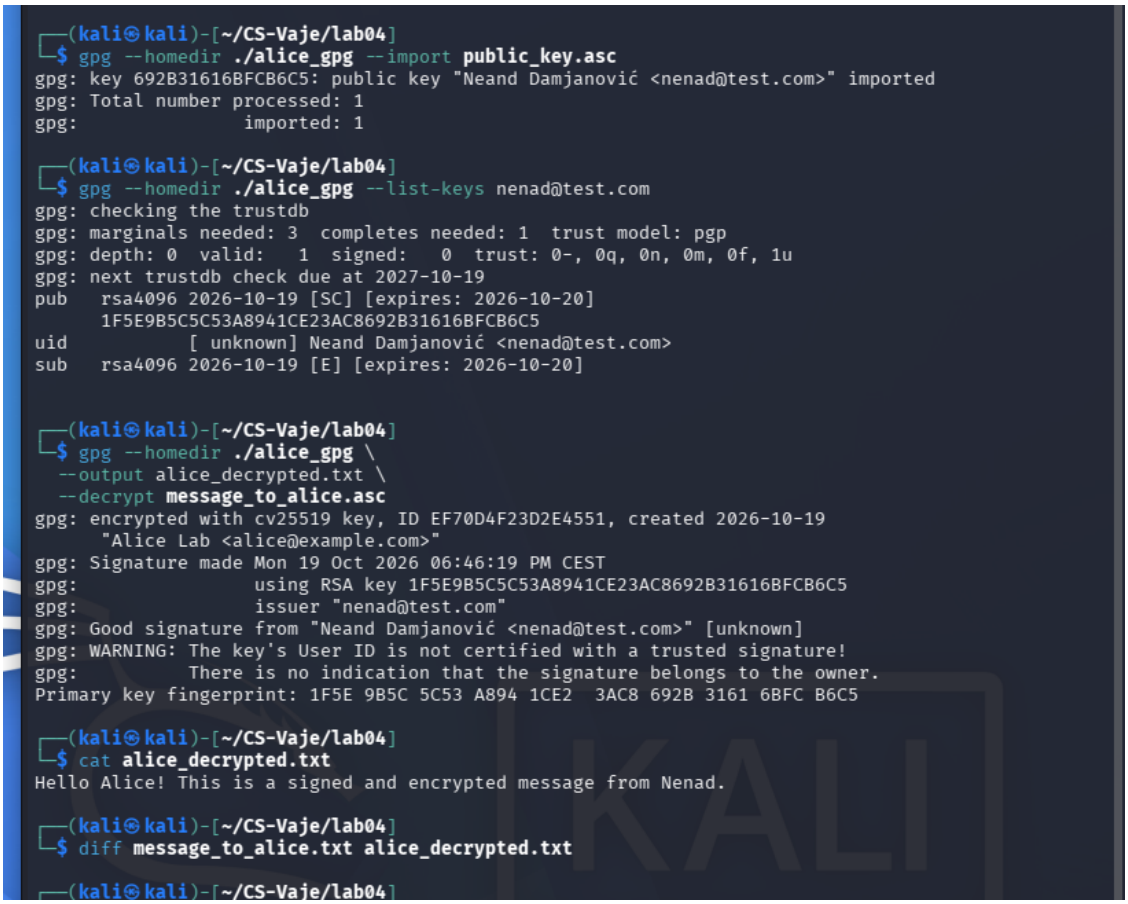
Figure 8. Encryption and signing for Alice



Recipient decryption and trust verification

Alice imported Nenad’s public key, decrypted the signed encrypted message in her own GPG environment, and received Good signature [unknown]. The plaintext matched the original by diff. The [unknown] trust label means the key-to-person identity was not independently certified; signature validity alone does not prove a real-world identity.

Figure 9. Alice decrypts and verifies the signed message



Questions and answers

Encryption vs. signing: Encryption provides confidentiality for the intended recipient. Signing supports verification of message integrity and the signing key, but does not hide content.

Role of keys: The recipient’s public key encrypts and their private key decrypts; the sender’s private key signs and the sender’s public key verifies. Public-key identity must be independently checked.

What if a file is modified? The demonstrated signed plaintext modification produced BAD signature. A separate direct corruption test of encrypted ciphertext was not performed.

Security and conclusion: All operations used local test identities. Never publish private keys, passphrases, the alice_gpg/ directory, or revocation certificates. GPG successfully demonstrated confidentiality, integrity checking, and signed encrypted communication.